

SEP WEEK 3: ARCHITECTURE & TECH STACKS

Alex Elwood, Sarah Connolly

Slides adapted from Daniel Schien

bristol.ac.uk



Overview

- System Architecture
 - Motivations for a formalization
 - Examples
- Modelling
 - Modelling systems at different levels
 - Static Modeling
 - Dynamic Modelling
- Tech Stacks
 - The technology which implements the system architecture
 - Examples from previous SEP projects
 - Choosing a tech stack
- Open Source & Software Licenses

Software Architecture – What

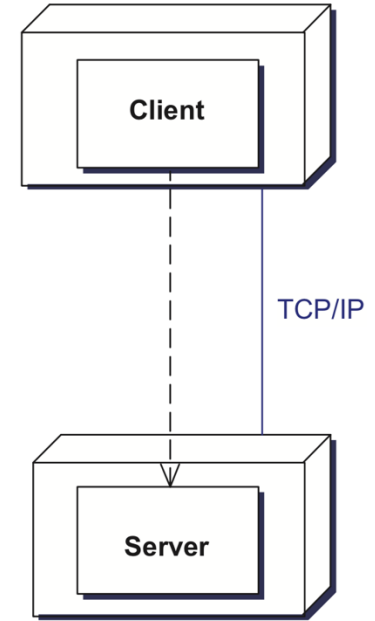
- Software architecture is a set of **structures** needed to **reason** about a system. It includes software components, the **relations** between them and properties of both.
- Architecture might be implicit
 - If you're building any significant system. You cannot not have an architecture. You might just not have documented or communicated explicitly.

Software Architecture – Why

- Enables the engineering of large systems
 - As systems grow it becomes unattainable to manage infrastructure without a well documented design
- Communication of Ideas
 - Between Developers
 - To non-technical stakeholders
- The role of a Systems Architect

Example 1: Client-Server

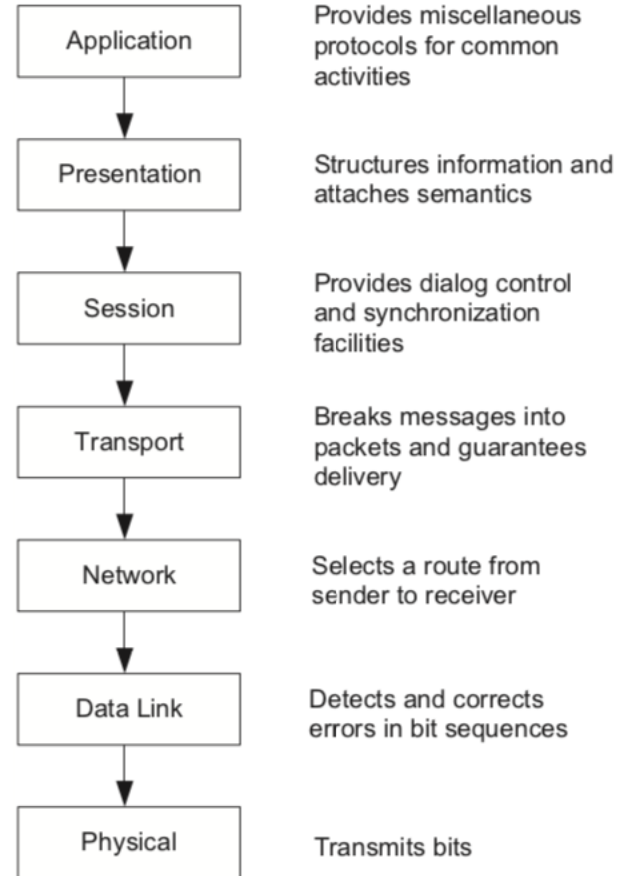
- clients requests services
- Servers are permanently listening
- Inter-process communication causes overhead
- Requests and result data often have to be transformed or marshalled
- sessions: stateless or stateful server



Example 2:

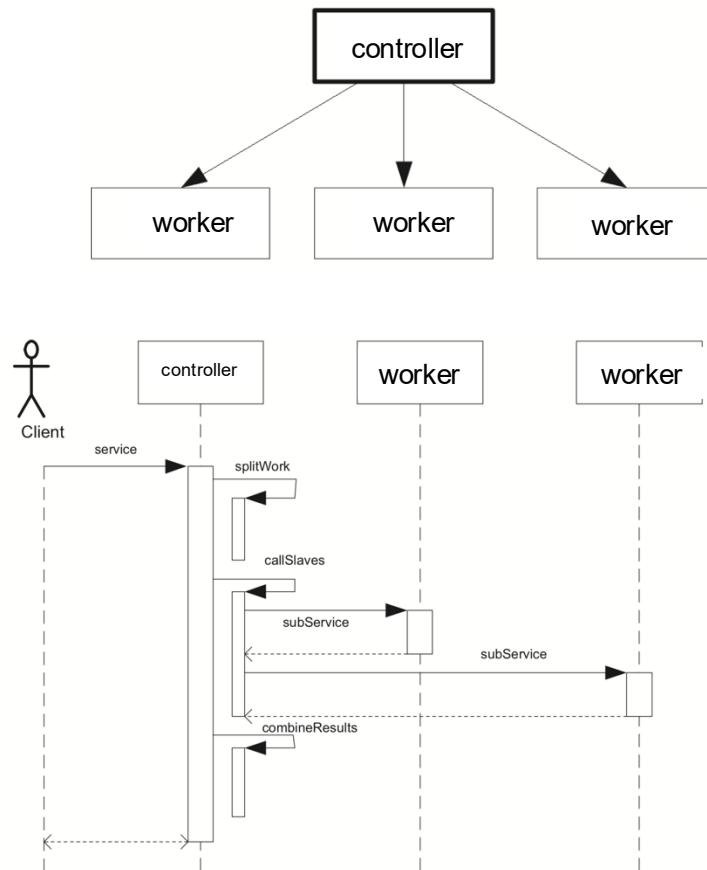
Layered Architecture

- lower layer can be used by different higher layers
- standardization
- Dependencies are kept local
- a change of the behaviour of a lower layer has an effect on the layers above it
- defining an abstract service interface is not an easy job
- lower layers may perform unnecessary work, not required by the higher layer.



Example 3: Controller / Worker

- divide-and-conquer
- Parallelisation, Fault Tolerance
- Most cloud infrastructure frameworks
 - Kubernetes, Hadoop, Spark
- Computer Systems A
- High Performance Computing



Software Modelling

Just Enough Modelling

- Big Design Upfront

*"Many times, **thinking things out in advance saved us serious development headaches later on.** ... [on making a particular specification change] ... Making this change in the spec took **an hour or two.** If we had made this change in code, it would have added **weeks** to the schedule. I can't tell you how strongly I believe in Big Design Up Front, which the proponents of Extreme Programming consider anathema. I have consistently saved time and made better products by using BDUF and I'm proud to use it, no matter what the XP fanatics claim. They're just wrong on this point and I can't be any clearer than that."*

Joel Spolsky, Stackoverflow and Trello

- Be Pragmatic: Design where it provides value
 - Break down complexity
 - Enable shared understanding
 - Driven by risk: how much would it hurt if you got it wrong?

Object Oriented Analysis and Design

- Analysis
 - Domain Modelling
 - Identify objects in the *problem domain*, relationships
- Design
 - Architecture
 - Static properties
 - Dynamic properties

Identifying Domain Entities

- Not mechanical
- Discussion Stakeholders
- Things to look out for
 - Objects
 - Business process
 - Job titles /roles
 - Sets, grouping, containers, elements
 - Events
 - Record of work

Entity associations

- Describe relationships
- Look out for
 - Part of
 - Description for
 - Set member
 - Uses/manages
 - Communicates with
 - Owned by
 - Event

Associations

- Can be named
 - Association name
 - Role names
- Can include multiplicity
- There can be more than one between two entities
- They can be recursive

Grammatical Parse

- Identify nouns from existing text
- Narrow down to remove
 - Duplicated variations
 - Irrelevant
 - Out of scope

Example

“Cloud computing is an information technology (IT) paradigm, a model for enabling ubiquitous access to shared pools of configurable resources (such as computer networks, servers, storage, applications and services),[1][2] which can be rapidly provisioned with minimal management effort, often over the Internet.”

https://en.wikipedia.org/wiki/Cloud_computing

Example

“Cloud computing is an information technology (IT) paradigm, a model for enabling ubiquitous access to shared pools of configurable **resources** (such as computer **networks**, **servers**, **storage**, **applications** and **services**),[1][2] which can be rapidly provisioned with minimal management effort, often over the **Internet**.”

https://en.wikipedia.org/wiki/Cloud_computing

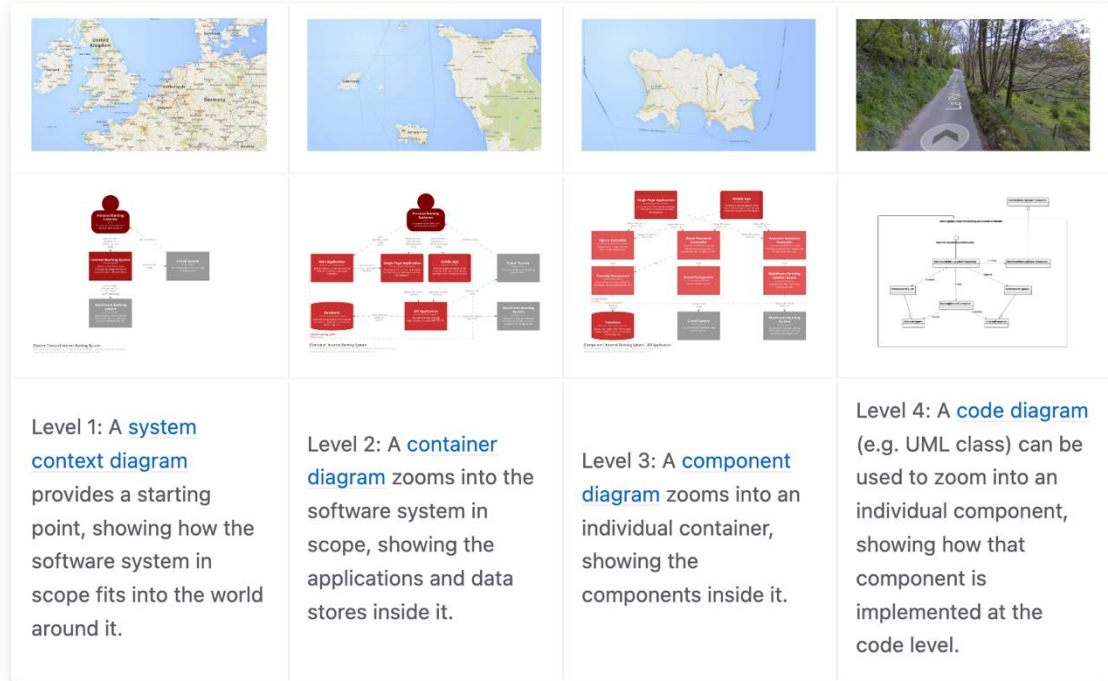
Task 1: Grammatically Analyse Your Project Brief

- As a team, read your project brief again. Parse the text to understand the objects involved.
- Look out for **objects** and their **associations** and write them down
- If the client has not included enough detail, then write your own brief which describes the objects and associations which might be involved
- 5-10 Minutes

Static Design Models

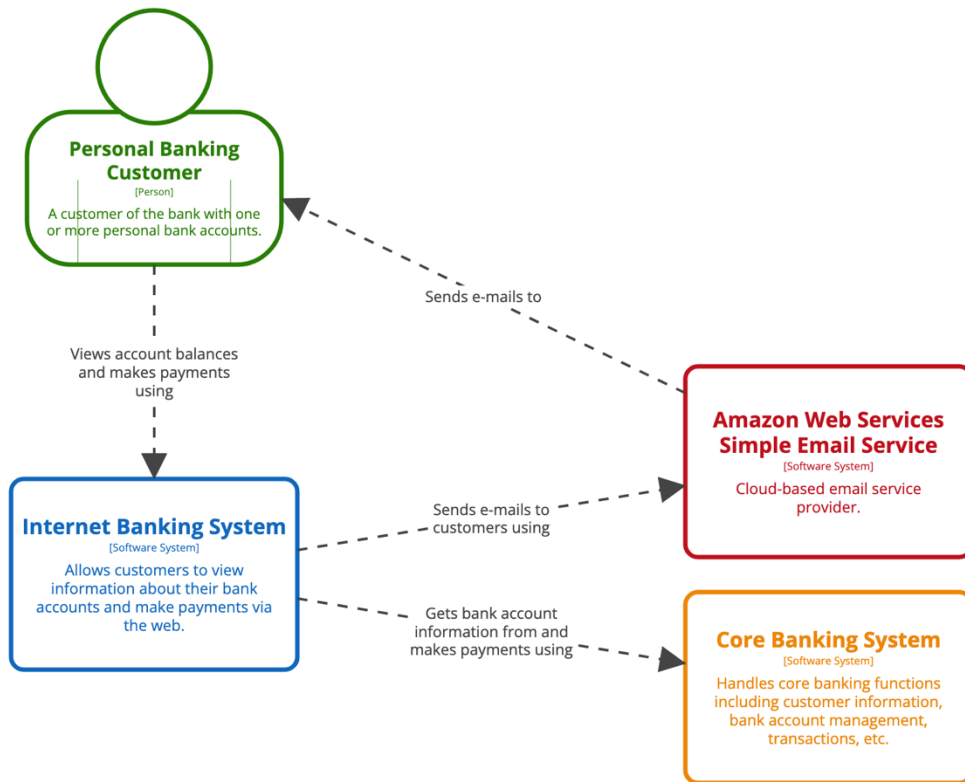
C4 model for visualising software architecture

- <https://c4model.com/introduction>
- Sequence of set choices of level of abstraction



C4 model - Level 1: System Context

- Intended to be used by technical and non-technical people
 - Useful for communicating the system back to the client

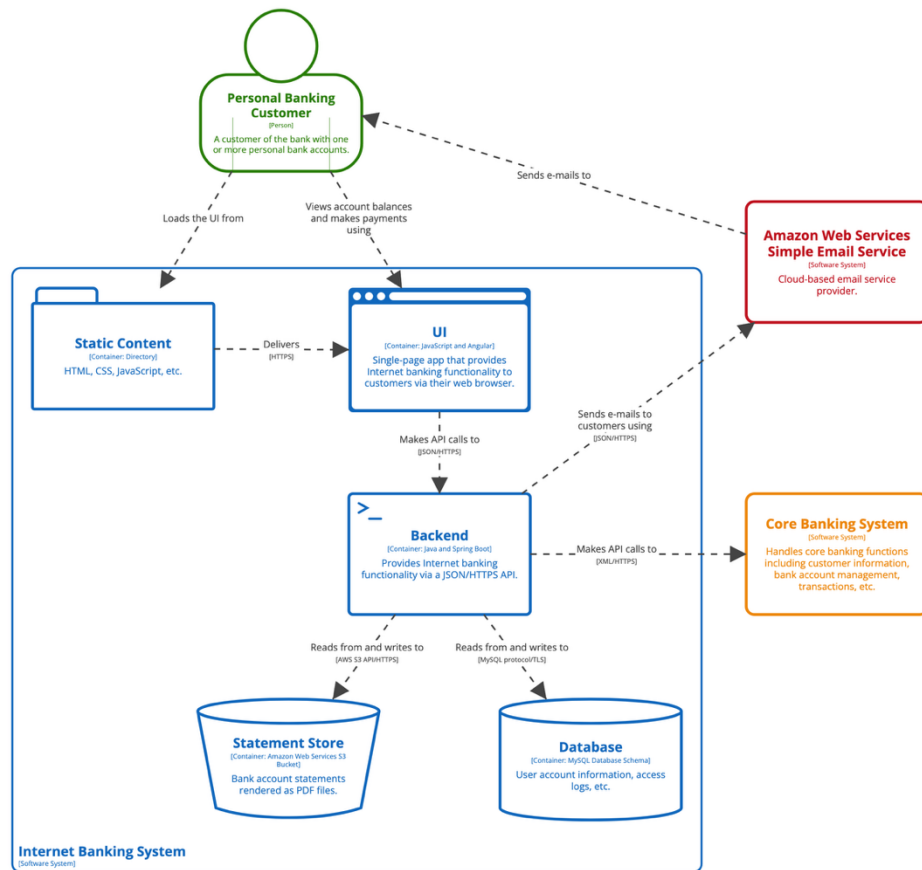


System Context View: Internet Banking System

The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

C4 model - Level 2: Container Diagram

- Intended for Technical people inside and outside the software development team; including software architects, developer and operations/support staff.

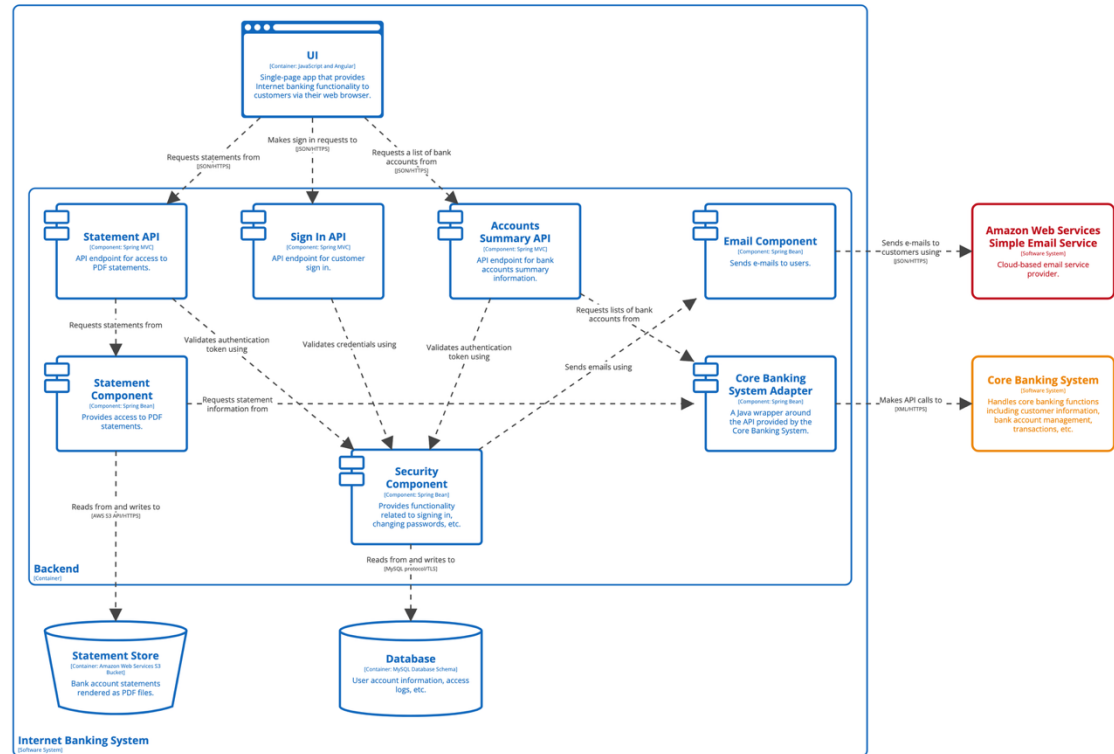


Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

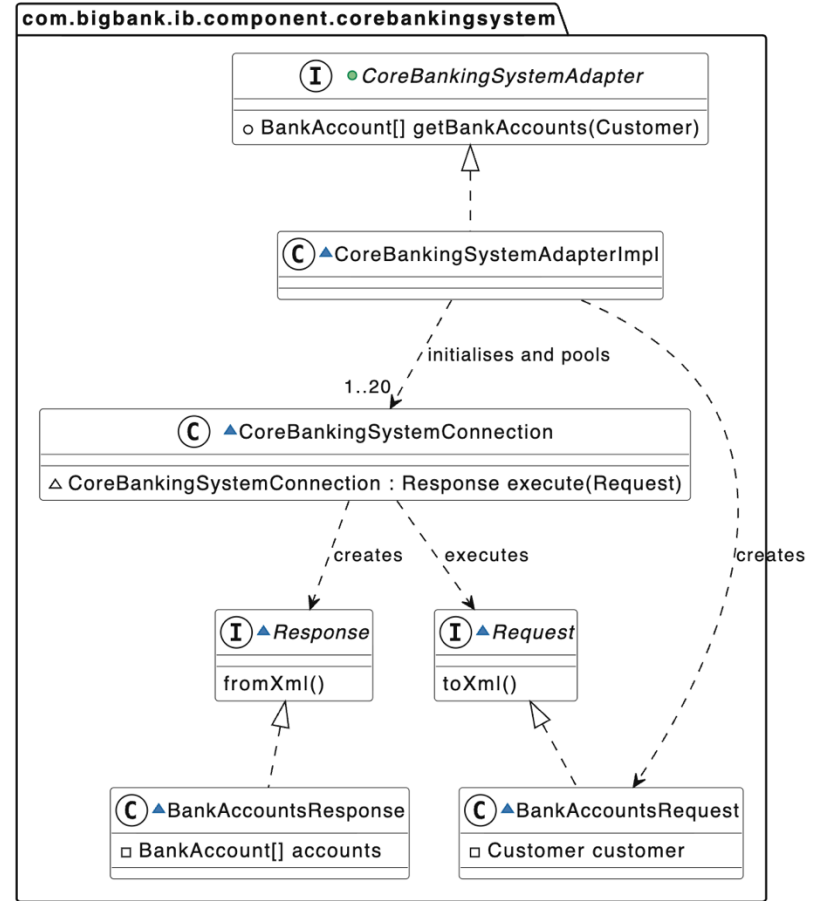
C4 model - Level 3: Component Diagram

- This could be:
 - A single element of your cloud infrastructure
 - The user application
- Intended to be used by software architects and developers
- Is it required?
 - Not needed for every part of the system
 - Recommended for more complex components
 - Useful for documentation and handover



C4 model - Level 4: Code Diagram

- A single component of the system
- Uses a formalized UML
- Not always necessary
 - Code structure can change frequently
 - Smart IDEs can generate this automatically



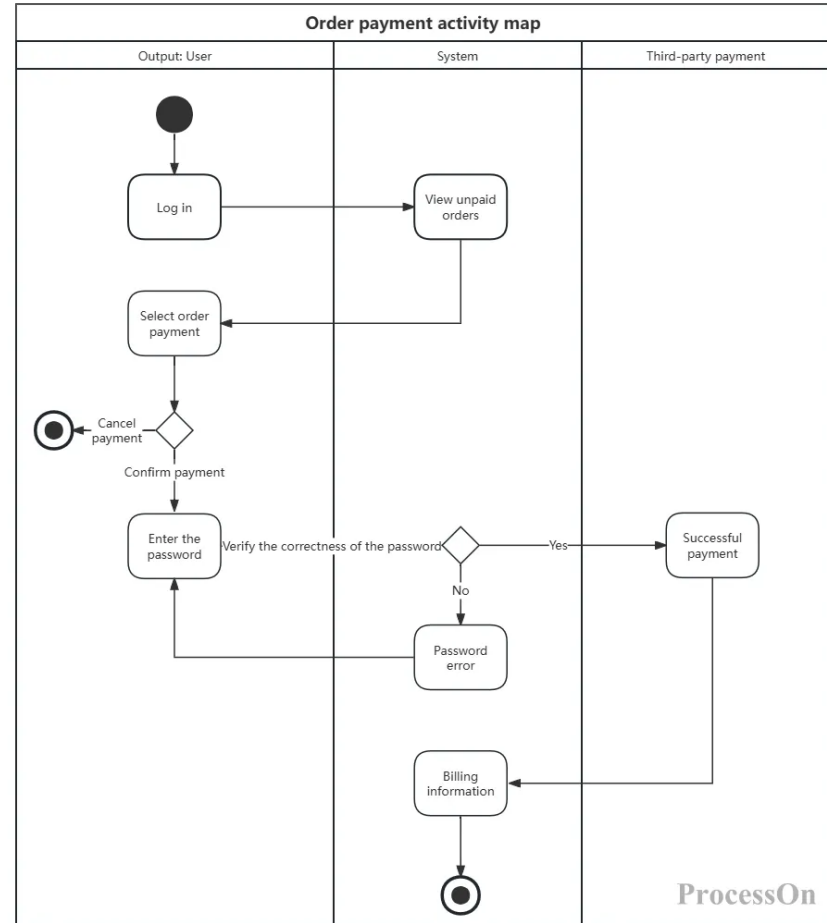
Task 2: Create a system context diagram

- Visit the C4 website for more info:
<https://c4model.com/introduction>
- As a team, create a system context diagram
- Keep it high level!
- Take a picture of it afterwards so your don't lose it!
- ~10-15 Minutes

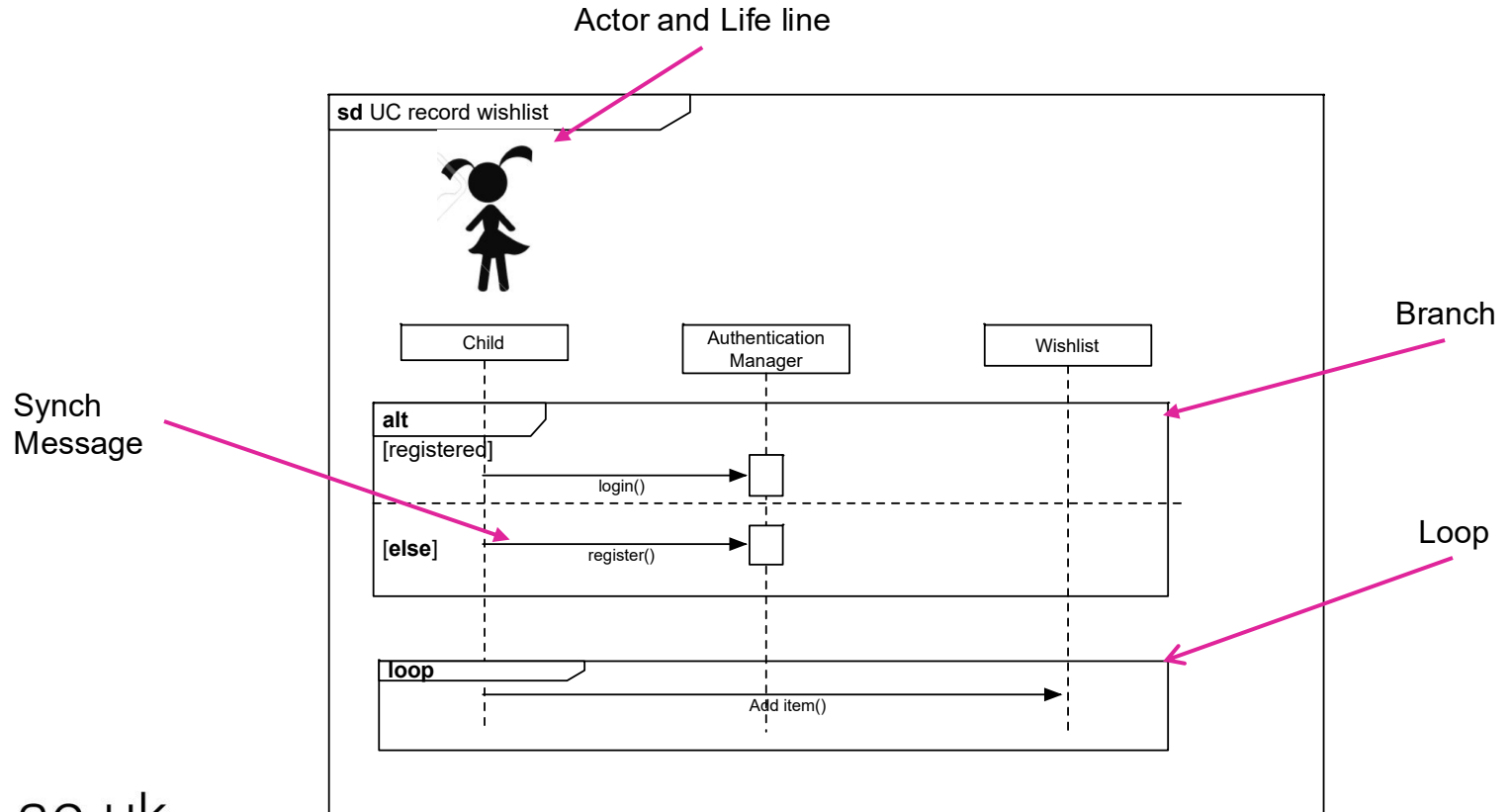
Dynamic Design Models

Activity Diagrams

- Defines an application workflow from start to finish including:
 - Starting and terminating points
 - Actions
 - Relations
 - Branches
 - Swimlanes



Sequence Diagram Example



Task 3: Create a sequence diagram

- Come up with scenarios of use for your project.
- Create a sequence diagram for each scenario
- ~10 Minutes

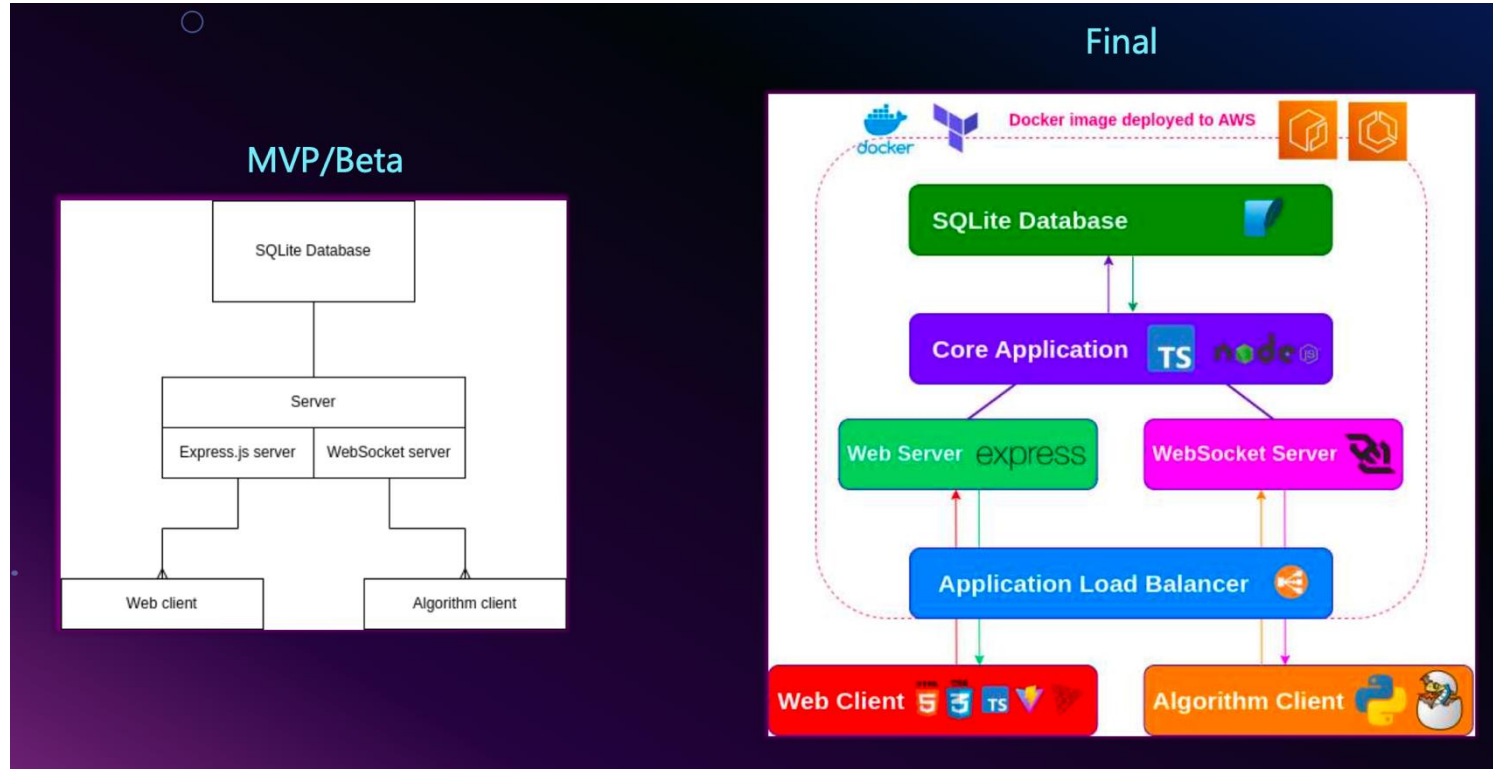
Choosing a Tech Stacks

Tech Stack Example 1: Web App

- **Languages:** JavaScript/TypeScript, HTML, CSS
- **Back End:** [Node.js](#)
- **Databases:** MySQL, PostgreSQL, MongoDB, and others
- **Targeted Platform:** Browser
- **Example Project:** [2023-RoboSim](#)
- or
- **Languages:** Java, JavaScript, HTML, CSS
- **Framework:** [Spring Boot](#) with [Thymeleaf](#)
- **Databases:** [Works with SQL or NoSQL databases](#)
- **Targeted Platform:** Browser
- **Example Project:** [2023-CityFarm](#)

Tech Stack Example 1:

Web App – 2024-RoboSim



Tech Stack Example 2:

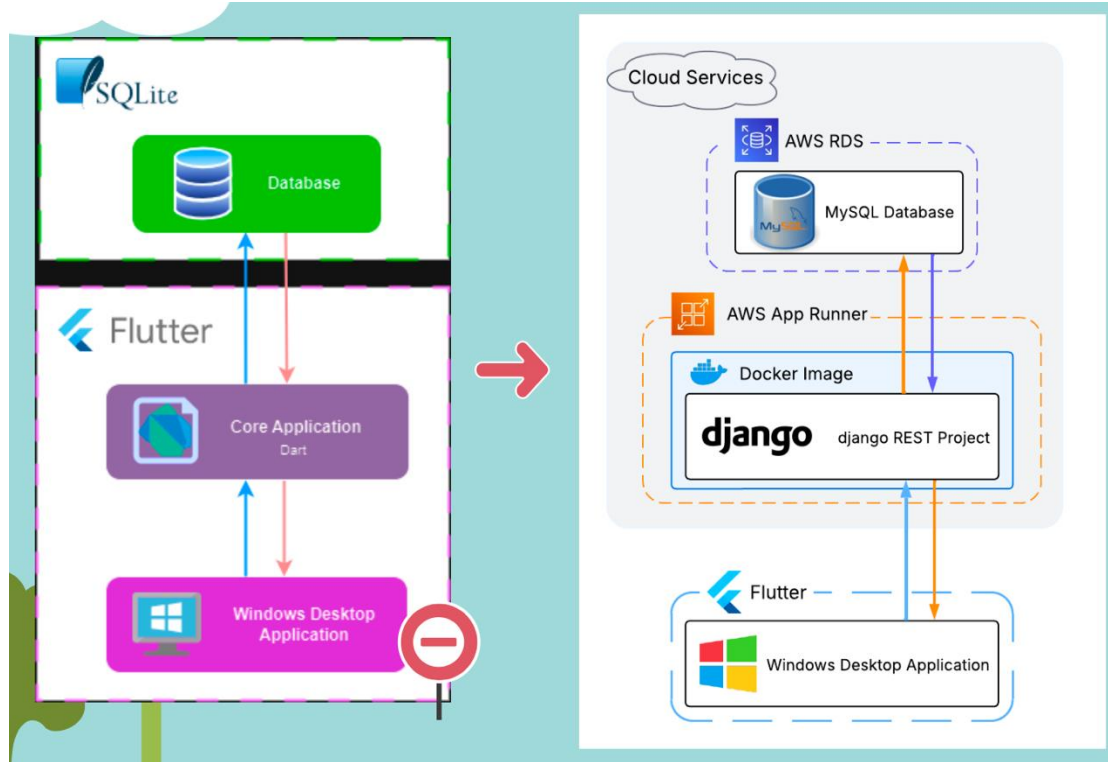
Cross-Platform / Mobile Applications

- Languages: [Dart](#)
- Framework: [Flutter](#)
- Databases: [PostgreSQL](#)
- Example Project: [2023-MarineConservationApp](#)

Tech Stack Example 2:

Cross-Platform / Mobile Applications

2024-SprintCommunityTransport

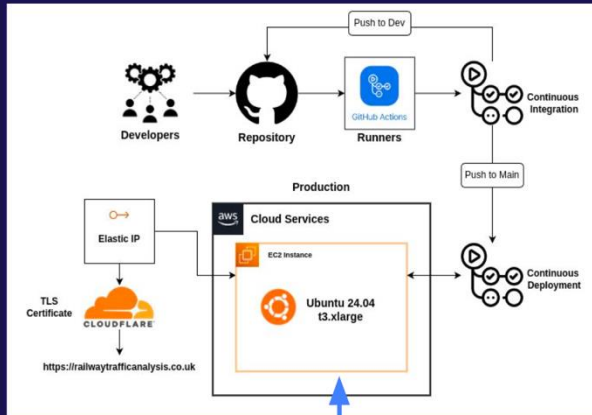


Tech Stack Example 2:

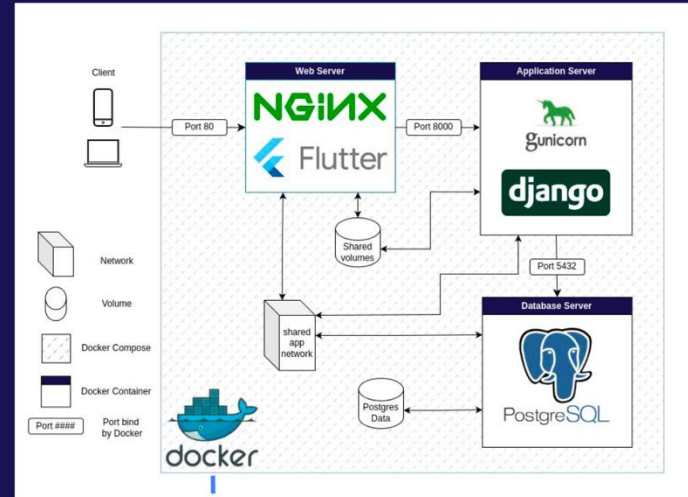
Cross-Platform / Mobile Applications

2024-RailwayTrafficAnalysis

CI/CD Workflow and AWS EC2 Deployment



Dockerized Web Application Architecture



Dockerized Web Application Running Inside EC2
(via Docker Compose)

Tech Stack Example 3:

Games & 3D Environments

- **Languages:** C#
- **Framework:** [Unity](#)
- **Databases:** C# libraries can connect to a variety of SQL or no-SQL databases.
- **Targeted Platforms:** Windows, Linux, macOS, iOS, Android and others.
- **Example Projects:** [2023-Edutopia](#) & [2023-SkillsLearningGame](#) (This has some interesting design documents in their 'docs' folders that you might want to take inspiration from.)

Tech Stack Example 4: Hardware Based Project

2024-LitterRecognition2

Tech Stack



Main Components

Deep Learning, Stereo Vision,
Robotics, Controller, User Interface



Languages



Python and C++ (Arduino)



Software



PyTorch

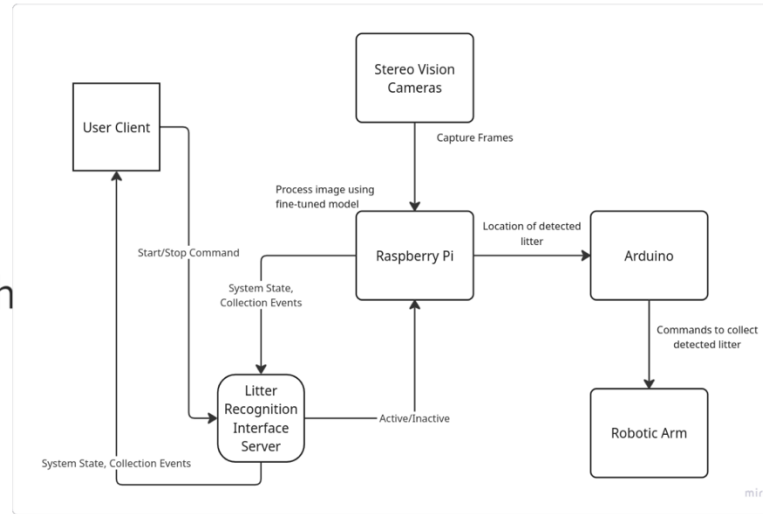
PyTorch, YOLO, OpenCV, AWS



Hardware



Raspberry Pi, Arduino, Robot
Arm, Dual Cameras



Middleware

- Software which connects different components or applications
- Sits between applications and infrastructure to simplify the handling of services such as:
 - Databases – e.g. JDBC
 - API Management – e.g. Nginx
 - Message Passing - e.g. Kafka
- Decouples components to increase scalability

Tech Stacks are not Static!

- Tech Stacks can:
 - Change
 - Evolve
 - Mature
 - Be rewritten

Task 3: Technology Choice Matrix

Tech Stack	Description	Fit to Project	Experience in Team	Maintenance	Risks
Fill in the matrix with evidence. Present to your client with advantages / disadvantages of each stack.	Where deployed? Vendor? Open Source? Maturity? Popularity?	What requirements drive this choice?	Have you worked with these technologies before? How high is the learning curve? Is there a champion who will support others? Where will you get help?	How will the client maintain the system built with these tech.?	Cost? Security/ software updates? Will the technologies interact well?
A					
B					
C					

Task 4: Create a Container Diagram

- Using the C4 model, zoom into one of your main components identified in tasks 1 & 2 and create a container diagram with the tech stack which most likely to be used
- 20 Minutes?

How to organise your team

- We DON'T recommend splitting up into a FrontEnd and BackEnd team, or similar
- While it's good to start out with specific tasks and roles, having static sub-teams is bad for SEP

How to organise your team

- **You will all need to review each others' code** - separation makes this much harder.
- **You will be completing work at a different pace** – leading to mismatched bursts
- **You will need to be integrating the work at some point**
- **Resilience:** If a team member leaves, your project shouldn't break
- **Flexibility is better for your goals** – in SEP and elsewhere

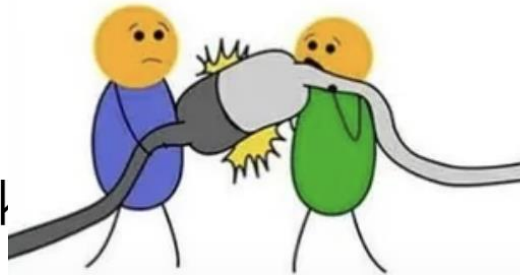
YOU WORK ON FRONT-END AND
I WILL WORK ON BACK-END.



AFTER A MONTH

FRONT-END

BACK-END



How to organise your team

- **DO NOT** organise sub-teams around nationality/language!
- You are at an international University, preparing for careers with international companies – employers will expect you to be able to work with people from different backgrounds/nationalities/languages
- **Rotating roles is a marks magnet!**

Open Source & Licenses

Software Licenses

- Licenses are not Patents!
- Copyright Laws
- Modification
- Distribution
- Usage – Commercial, Patent, Private
- Liability – Can you be held responsible if something goes wrong?
- Warranty – Will you help if something goes wrong?
- Working within software communities
 - i.e. a Wordpress Plugin must have a GNU GPLv2 license or later

Open Source Software

- Improves collaboration & compatibility
- Community Driven
- Most SEP projects are Open Source
- Comes with a legal responsibility to ensure other licenses are not broken
 - Are you using any libraries in accordance with their license?
 - What are the licenses restrictions of any images or resources?
- Discuss with your client!

Task 5: Choose a License

- Discuss as a group:
 - Will the software be closed or open source?
 - Which license will the project have - [Choose-a-License](#)
- Push a LICENSE file to the root directory of your project
- If intending to become open source: Explore the [FOSSA tool](#)

You might need to discuss this with your client at the next meeting!

Wrapping Up!

- Independent Software Development Research
- Expectations for SEP:
 - Develop a software system of significant scale within a mature and repeatable Software Development process and apply Agile Principles & Practices.
 - Select and employ development support tools (as appropriate to the project being undertaken).

Questions?